

TP 3 : Métriques d'évaluation d'un classifieur supervisé et jeux de données d'entraînement et de test

Le but de ce TP va être :

- D'écrire des fonctions permettant de calculer l'accuracy et les autres métriques vues en cours pour évaluer les performances d'un classifieur
- De voir comment partager un jeu de données en plusieurs portions afin d'effectuer l'apprentissage et l'évaluation du classifieur

Pour simplifier nous allons travailler avec un classifieur binaire simple (le modèle de régression logistique, que nous étudierons plus en détails dans le cours si nous avons le temps) et n'utiliser que certaines de nos variables pour prédire la survie au naufrage. Dans les prochains TP nous nous concentrerons sur les classifieurs étudiés en cours, et nous les entraînerons sur le jeu de données Titanic complet (ou sur votre propre jeu de données)

1. Jeu de données

On charge le jeu de données obtenu en fin de TP 2 :

```
library(dplyr)
library(tidyverse)
data = read.csv("titanic_pre_processed.csv", stringsAsFactors = TRUE)
y = data$Survived # notre target
X = data %>% select(Fare, Fam) # pour simplifier on n'utilise que deux de nos variables explicatives
```

2. Un classifieur simple

Nous utiliserons le classifieur binaire implémenté ci-dessous :

```
myClassifieur = function (X_train, y_train, X_test, threshold = 0.5){
  data_train = cbind(y_train, X_train)
  mod = glm(y_train ~ ., data = data_train, family = binomial(link = "logit"))#explain la syntaxe
  y_pred = 0 + 1 * (predict.glm(mod, newdata = X_test, type = "response") > threshold)
  return(y_pred)
}
```

L'apprentissage du modèle de régression logistique se fait sur `X_train` et `y_train`. En sortie de la fonction `myClassifieur` nous obtenons `y_pred`, le vecteur contenant les catégories prédites pour chacune des observations dans `X_test`. En réalité, le classifieur prédit d'abord pour chaque observation dans `X_test` la probabilité d'appartenance à la classe positive. Cette probabilité est comparée à la valeur seuil `threshold` pour enfin prédire la catégorie la plus probable.

Quelle est la valeur seuil du classifieur par défaut ?

Réponse : 0.5

Par exemple, on peut entraîner le classifieur sur les données complètes, puis prédire la target des 10 premières observations du jeu de données, pour une valeur seuil de 0.5 et pour une valeur seuil de 0.3

Réponse :

```

y_pred1 = myClassifier(X_train = X, y_train = y,
                      X_test = X[1:10,], threshold = 0.5)
y_pred2 = myClassifier(X_train = X, y_train = y,
                      X_test = X[1:10,], threshold = 0.3)
y[1:10]

```

```
## [1] 0 1 1 1 0 0 0 0 1 1
```

```
y_pred1
```

```
##  1  2  3  4  5  6  7  8  9 10
##  0  1  0  0  0  0  0  0  0  0
```

```
y_pred2
```

```
##  1  2  3  4  5  6  7  8  9 10
##  1  1  1  1  1  1  1  0  0  1
```

Que remarque-t-on ? plus le treshhold est bas, plus on prédit de 1 *Réponse :*

3. Ensembles d'apprentissage et de test - méthode holdout

Que font les lignes de code ci-dessous ?

```

n      = nrow(data)
inTrain = sample(1:n, size = round(0.8 * n), replace = FALSE)
X_train = X[inTrain,]
y_train = y[inTrain]
X_test  = X[-inTrain,]
y_test  = y[-inTrain]

```

Réponse :

Remarque : si `y` était un dataframe (par exemple si on utilise le fichier `titanic_pre_processed_target.csv`), alors il faudrait utiliser `y_train = y[inTrain,]` et `y_test = y[-inTrain,]`

Écrire une fonction nommée `holdOut` qui implémente la méthode du holdout pour partager nos données en un ensemble d'apprentissage et un ensemble de test. Penser à créer un argument `split` permettant de spécifier le split train/test souhaité

Réponse :

```

#holdOut = function(data, split = 0.8){
#  y = data$Survived # notre target
#  X = data %>% select(Fare, Fam)
#  return(holdOut(X, y, split))
#}
holdOut = function(X,y, split = 0.8){
  n      = nrow(X)
  inTrain = sample(1:n, size = round(split * n), replace = FALSE)

```

```

X_train = X[inTrain,]
y_train = y[inTrain]
X_test  = X[-inTrain,]
y_test  = y[-inTrain]

return(list(X_train = X_train, y_train = y_train,
           X_test = X_test, y_test = y_test))
}

#which ?   X_train = data[inTrain, -which(names(data) == "Survived")]; list

```

Que fait la ligne de code ci-dessous ? À quoi cela peut-il servir ?

```
set.seed(1234)
```

Réponse : Cela permet de fixer la graine du générateur de nombres aléatoires, afin que les résultats soient reproductibles

Utiliser la fonction `holdOut` sur le jeu de données `data` avec un split train/test de 80/20

Réponse :

```
data_split = holdOut(X=X,y=y, split = 0.8)
```

Sinon on pourrait utiliser la commande `createDataPartition` du package `caret`

```
install.packages("caret")
```

```
library(caret)
```

```
## Loading required package: lattice

##
## Attaching package: 'caret'

## The following object is masked from 'package:purrr':
##
##      lift

```

```
createDataPartition(y = y, p = 0.8)
```

Elle retourne les indices des individus qui sont dans l'ensemble d'apprentissage. C'est l'équivalent du `inTrain` retourné par ma fonction `holdOut`

Utiliser `myClassifieur` pour prédire la target des individus dans le jeu de données de test avec la valeur par défaut du seuil et après apprentissage sur les individus dans le jeu de données train

Réponse :

```

y_pred = myClassifieur(X_train = data_split$X_train, y_train = data_split$y_train,
                      X_test = data_split$X_test)
#data_split$y_test
y_pred

```

```
## 5 8 9 16 18 20 26 44 46 47 50 56 62 68 69 72 73 78 80 81
## 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0
## 89 90 92 93 96 99 109 111 115 117 129 130 134 137 139 140 144 148 151 158
## 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0
## 167 172 173 175 180 193 196 197 200 209 213 217 219 220 233 236 239 240 242 244
## 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## 246 247 266 268 273 282 287 288 290 300 301 304 315 320 323 325 333 336 342 351
## 0 0 0 1 1 0 1 1 0 0 1 0 1 0 0 0 1 0 0 0
## 355 356 360 366 369 370 373 374 385 391 393 396 406 407 411 415 416 435 437 438
## 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0
## 443 446 452 453 458 467 477 483 492 493 496 503 506 507 514 517 532 536 549 553
## 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0
## 556 559 575 579 603 610 616 619 622 625 631 633 655 657 662 672 674 677 684 686
## 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0
## 689 690 692 693 695 698 702 709 710 711 714 715 718 723 725 730 734 735 749 756
## 0 1 0 0 0 1 0 0 0 0 0 0 0 0 0 1 1 0 0 0
## 762 765 769 771 785 797 805 815 818 830 837 845 857 859 864
## 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0
```

4. Métriques d'évaluation

4.1. Accuracy, recall, precision, F1score

Écrire 3 fonctions nommées `myAccuracy`, `myRecall`, et `myPrecision`, et qui calculent les métriques comme définies en cours, puis évaluer les performances de `myClassifier` sur la partie test du jeu de données

Rappel : Ces métriques sont basées sur la comparaison de `y_pred` et `y_test`

Réponse :

Installer puis charger le package `Metrics`. Quelles fonctions vous semblent intéressantes pour ce que nous souhaitons faire ? Vérifier que vos fonctions créées ci-dessus vous retournent les mêmes résultats

Réponse :

```
library(Metrics)
detach("package:caret", unload = TRUE)
```

Commenter les valeurs de ces métriques

Réponse :

Le package `caret` propose aussi des fonctions permettant de calculer certaines de ces métriques. Lire la documentation de la commande `recall` du package `caret` ; quel semble être son avantage par rapport à la commande `recall` du package `Metrics` ou à celles que nous avons écrite ?

Réponse :

Commenter le choix des noms des fonctions implémentant les métriques de performance dans les packages `caret` et `Metrics`

Réponse :

Le package `caret` permet aussi d'obtenir la matrice de confusion avec la commande `confusionMatrix`

```
library(caret)
confusionMatrix(data = factor(y_pred), reference = factor(data_split$y_test),
  positive = "1")$table
```

Le jeu de données est-il bien équilibré ?

Réponse :

Lire la partie Details de la documentation de la commande `confusionMatrix`. À quelles métriques vues en cours peut-on en fait avoir accès ?

Réponse :

En utilisant le package `caret`, donner la valeur de la spécificité et de la balanced accuracy. Commenter

Réponse :

Prédire à nouveau la target mais pour différentes valeurs de `threshold` dans la fonction `myClassifier`. Quel est l'impact sur le recall et la précision ?

Réponse :

Que font les lignes de code ci-dessous ? Que remarque-t-on ?

```
y_pred3 = myClassifier(X_train = data_split$X_train, y_train = data_split$y_train,
                      X_test = data_split$X_train, threshold = 0.5)
myRecall(data_split$y_train, y_pred3)
myPrecision(data_split$y_train, y_pred3)

myRecall(data_split$y_test, y_pred)
myPrecision(data_split$y_test, y_pred)
```

Réponse :

4.2. AUC

Modifier `myClassifier` pour créer une fonction nommée `myClassifier2` qui, au lieu de retourner la catégorie d'appartenance, retourne la probabilité prédite d'appartenir à la catégorie positive

Réponse :

Prédire la probabilité d'appartenance à la catégorie positive pour les individus dans le jeu de données de test

Réponse :

Installer puis charger le package `pROC`. Tracer la courbe ROC et donner l'AUC du classifieur `myClassifier2` sur la partie test du jeu de données

Réponse :

```
install.packages("pROC")

library(pROC)
```

Commenter la forme de la ROC curve, la valeur de l'AUC obtenue, ainsi que le nom de l'axe des ordonnées

Réponse :

Remarque : on peut aussi obtenir l'AUC à partir de la commande `roc`, en spécifiant l'argument `auc = TRUE`

5. Ensembles d'apprentissage et de test - k-fold cross validation

Que font les lignes de code ci-dessous ?

```
set.seed(1234)
nfolds = 10
folds = NULL
for(i in 1:n){
  folds = c(folds, sample(1:nfolds, 1, replace = FALSE))
}
```

Réponse :

Ou alors on pourrait écrire une fonction qui diviserait aléatoirement le jeu de données en k portions disjointes, et retournerait les indices des individus qui sont dans chacun des k folds, comme le fait la fonction `createFolds` du package `caret`

Que fait la fonction CV donnée ci-dessous ?

```
CV = function(X, y, folds){
  nfolds      = max(folds)
  accuracy_k  = numeric(nfolds)
  recall_k    = numeric(nfolds)
  for(k in 1:nfolds){
    inFold     = which(folds == k)
    X_train    = X[-inFold,] # pour entraîner le modèle
    X_test     = X[inFold,]  # pour évaluer le modèle
    y_train    = y[-inFold]
    y_test     = y[inFold]

    y_test_pred = myClassifier(X_train = X_train, y_train = y_train,
                              X_test = X_test, threshold = 0.5)

    accuracy_k[k] = myAccuracy(y_test, y_test_pred)
    recall_k[k]   = myRecall(y_test, y_test_pred)
  }
  return(list(accuracy_k = accuracy_k, recall_k = recall_k ))
}
```

Réponse :

Utiliser de manière répétée les différentes portions du jeu de données pour entraîner et évaluer le classifieur `myClassifier`, puis calculer l'accuracy et le recall moyen sur les 10 folds

Remarque : Une alternative serait de créer une fonction prenant en argument `X`, `y`, et le nombre de folds souhaités pour la cross validation. L'assignation des observations aux folds se ferait alors à l'intérieur de la fonction

```
CV = function(X, y, nfolds){
  n = nrow(X)
  folds = NULL
  for(i in 1:n){
    folds = c(folds, sample(1:nfolds, 1, replace = FALSE))
  }

  accuracy_k = numeric(nfolds)
  recall_k   = numeric(nfolds)
  for(k in 1:nfolds){
    inFold     = which(folds == k)
```

```
X_train      = X[-inFold,]  # pour entraîner le modèle
X_test       = X[inFold,]   # pour évaluer le modèle
y_train      = y[-inFold]
y_test       = y[inFold]

y_test_pred = myClassifier(X_train = X_train, y_train = y_train,
                           X_test = X_test, threshold = 0.5)

accuracy_k[k] = myAccuracy(y_test, y_test_pred)
recall_k[k]   = myRecall(y_test, y_test_pred)
}
return(list(accuracy_k = accuracy_k, recall_k = recall_k ))
}
```

6. Prochaines étapes

On pourra utiliser nos fonctions `holdOut` et `CV`, ou celles du package `caret`, pour partager le jeu de données en train et test

On pourra utiliser nos fonctions `myAccuracy`, `myRecall`, `myPrecision` et `myF1score`, ou celles du package `Metrics` pour évaluer les classifieurs que l'on considèrera